Space Empire

Supplément — Structure et organisation du code

Complément au document de conception technique v1

Mars 2026

1. Vue d'ensemble du projet

Le projet suit la structure conventionnelle Symfony. Les dossiers standards (Controller/, Entity/, Repository/) restent plats — un seul niveau, tous les fichiers du même type ensemble. Seul Service/ est organisé en sous-dossiers par domaine fonctionnel, car sans ça il deviendrait rapidement illisible avec 30+ services.

```
project/
├── config/
│   ├── game/                    # Catalogues YAML (données statiques de game
design)
│   ├── packages/                # Configuration des bundles Symfony
│   └── services.yaml            # Déclaration et injection des services
│
├── src/
│   ├── Controller/              # Points d'entrée HTTP
│   ├── Entity/                  # Entités Doctrine (état persisté en BDD)
│   ├── Integration/             # Services externes (S3, Discord, Mercure...)
│   ├── Manager/                 # Liaison entités ↔ repositories (persistance)
│   ├── Message/                 # DTOs des messages RabbitMQ
│   ├── MessageHandler/          # Consumers RabbitMQ (Symfony Messenger)
│   ├── Repository/              # Requêtes BDD (Doctrine)
│   ├── Service/                 # Logique métier, organisée par domaine
│   ├── ValueObject/             # Objets immuables sans persistance BDD
│   └── Validator/
│       └── DTO/                 # Validation et normalisation des données
entrantes
│
├── migrations/                  # Migrations Doctrine
└── tests/
    ├── Service/                 # Tests unitaires
    └── Controller/              # Tests fonctionnels
```

1.1 Règles fondamentales

- Un Controller ne contient jamais de logique métier. Il valide la requête, appelle un service, retourne une réponse HTTP.
- Un Service ne retourne jamais une JsonResponse. Il ne connaît pas HTTP.
- Un Manager encapsule la persistance : il crée, modifie et sauvegarde les entités via les repositories. Il ne contient pas de règles métier.
- Une Entity ne charge jamais ses données depuis un catalogue YAML. Elle stocke un slug (string) qui fait référence au catalogue.
- Un MessageHandler est fin : il orchestre les services, ne contient pas de logique propre.
- Les services dans Integration/ ne connaissent pas le jeu. Ils savent uniquement parler à un service externe.

2. config/game/ — Catalogues YAML

Contient toutes les données statiques de game design. Ces fichiers sont versionnés dans Git comme du code source — toute modification est donc tracée (qui, quoi, quand, pourquoi). Aucune donnée de catalogue ne va en base de données.

```
config/game/
├── ships/
│   ├── fighter.yaml
│   ├── bomber.yaml
│   ├── interceptor.yaml
│   ├── stealth_fighter.yaml
│   └── ...
├── buildings/
│   ├── mine_metal.yaml
│   ├── shipyard.yaml
│   ├── lab.yaml
│   └── ...
├── technologies/
│   ├── advanced_propulsion.yaml
│   ├── precision_fire.yaml
│   └── ...
├── defenses/
│   ├── ground_turret.yaml
│   └── ...
└── combat_modifiers.yaml          # Matrice attaquant vs défenseur
```

Dossier / Fichier	Contenu
ships/	Un fichier par vaisseau. Stats (hp, precision, tracking...), coûts de construction, prérequis de déblocage.
buildings/	Un fichier par type de bâtiment. Coûts, temps de construction, effets par niveau, prérequis par palier.
technologies/	Un fichier par technologie. Prérequis, coûts, temps de recherche, effet (unlock ou stat_bonus).
defenses/	Un fichier par type de défense statique. Mêmes stats que les vaisseaux (hp, precision, etc.).
combat_modifiers.yaml	Matrice globale des modificateurs de dégâts entre catégories de vaisseaux.

> *Modifier un fichier YAML en production a un effet immédiat sur tous les calculs futurs (construction, combat, recherche). Les vaisseaux déjà construits utilisent les nouvelles stats dès leur prochain combat. Ce comportement est volontaire et documenté.*

3. src/Controller/

Points d'entrée HTTP de l'application. Chaque controller gère un domaine fonctionnel. Structure interne d'une méthode de controller :

- Désérialiser et valider la requête via un DTO (Validator/DTO/)
- Appeler le service métier correspondant
- Retourner une JsonResponse

```
Controller/
├── FleetController.php          # Envoi de flotte, changement de vitesse, retour
├── ColonyController.php         # Gestion des ressources, bâtiments,
colonisation
├── ShipyardController.php       # File de construction des vaisseaux
├── ResearchController.php       # Lancement et suivi des recherches
├── CombatController.php         # Consultation des logs de combat
├── PlayerController.php         # Profil, alliances, contrats
└── GuildController.php          # Gestion des guildes et relations inter-guildes
```

```
// Exemple de structure d'une méthode controller
public function sendFleet(Request $request): JsonResponse
{
    // 1. Validation
    $dto    = new SendFleetDTO($request->toArray());
    $errors = $this->validator->validate($dto);
    if (count($errors) > 0) {
        $errorResponse = new ErrorResponse($errors);
        return new JsonResponse($errorResponse->getPayload(), $errorResponse-
>httpStatus);
    }

    // 2. Appel service métier
    $fleet = $this->fleetService->sendFleet($dto, $this->getUser());

    // 3. Réponse
    return new JsonResponse(
        $this->normalizer->normalize($fleet, 'json', ['groups' =>
['fleet_show']]),
        Response::HTTP_CREATED
    );
}
```

4. src/Entity/

Entités Doctrine. Chaque entité représente une table en base de données. Ce sont les objets qui persistent l'état dynamique du jeu — ce qui change au fil du temps (positions des flottes, stocks de ressources, files de construction, etc.).

Ce qui NE va PAS en Entity : les données statiques de catalogue (stats des vaisseaux, coûts des bâtiments...). Ces données sont dans les YAML et chargées via les Catalog services.

```
Entity/
├── — Univers ──────────────────────────────────────
│   ├── Galaxy.php                # Une galaxie (nom uniquement)
│   ├── SolarSystem.php           # Système stellaire (coordonnées X,Y dans sa
galaxie)
│   └── Planet.php                # Planète (position 1-9, statut, propriétaire)
│
├── — Joueur & relations ────────────────────────────
│   ├── Player.php                # Compte joueur
│   ├── Guild.php                 # Guilde
│   ├── GuildRelation.php         # Statut entre deux guildes
(neutre/alliés/ennemis)
│   ├── AllianceContract.php      # Contrat d'alliance direct joueur↔joueur
│   └── PlayerDiscovery.php       # Fog of War : planètes découvertes par un
joueur
│
├── — Colonie ──────────────────────────────────────
│   ├── PlanetBuilding.php        # Bâtiment construit sur une planète (slug +
niveau)
│   ├── PlanetResource.php        # Stock de ressource sur une planète (lazy calc)
│   └── PlanetDefense.php         # Défenses statiques sur une planète (slug +
quantité)
│
├── — Technologies ─────────────────────────────────
│   ├── PlayerTechnology.php      # Technologie débloquée par un joueur (slug +
niveau)
│   └── ResearchQueue.php         # Recherche en cours sur une planète
│
├── — Flotte & construction ────────────────────────
│   ├── Fleet.php                 # Flotte (statut, cargo, vitesse, dates)
│   ├── FleetShip.php             # Ligne de flotte (slug + quantité)
│   ├── Shipyard.php              # Chantier spatial d'une planète (niveau)
│   └── ShipyardQueue.php         # Vaisseau en cours de construction
│
└── — Combat ───────────────────────────────────────
    ├── CombatEvent.php           # Événement de combat (timer, statut)
    ├── CombatParticipant.php     # Participant à un combat (rôle, round d'entrée)
    └── CombatLog.php             # Log généré après résolution
```

4.1 Règle importante — les slugs

Les entités qui référencent un catalogue YAML stockent uniquement le slug (string), jamais l'objet de définition :

```
// Dans FleetShip.php
#[ORM\Column(type: 'string')]
private string $shipTypeSlug;    // 'fighter', 'bomber_mk2', etc.

// Pour obtenir les stats du vaisseau, on passe par le catalogue dans le service :
// $definition = $shipCatalog->get($fleetShip->getShipTypeSlug());
```

5. src/Repository/

Un repository par entité. Contient toutes les requêtes BDD spécifiques au domaine concerné. Les méthodes findBy* simples sont héritées de ServiceEntityRepository — on n'écrit que ce qui nécessite du DQL ou QueryBuilder.

```
Repository/
├── GalaxyRepository.php
├── SolarSystemRepository.php
├── PlanetRepository.php
├── PlayerRepository.php
├── GuildRepository.php
├── AllianceContractRepository.php
├── PlanetResourceRepository.php
├── PlayerTechnologyRepository.php
├── ResearchQueueRepository.php
├── FleetRepository.php
├── ShipyardRepository.php
├── ShipyardQueueRepository.php
└── CombatEventRepository.php
```

> *Les services injectent les repositories dont ils ont besoin. Un service ne fait jamais de requête SQL directement — il passe toujours par un repository.*

6. src/Manager/

Les managers sont la couche de liaison entre les services métier et les repositories. Ils encapsulent toutes les opérations de persistance : création, modification et sauvegarde des entités. Ils ne contiennent pas de règles métier — c'est le rôle des services.

```
Manager/
├── FleetManager.php             # Création et mise à jour des flottes et FleetShip
├── PlanetManager.php            # Colonisation, mise à jour du statut des planètes
├── PlayerManager.php            # Création et gestion des joueurs
├── GuildManager.php             # Création et gestion des guildes et relations
├── ColonyManager.php            # Bâtiments, ressources, défenses sur une planète
├── ShipyardManager.php          # File de construction (ShipyardQueue)
├── ResearchManager.php          # File de recherche (ResearchQueue, PlayerTechnology)
└── CombatManager.php            # CombatEvent, CombatParticipant, CombatLog
```

6.1 Responsabilité d'un Manager

Couche	Fait quoi	Ne fait PAS
Service	Règles métier, calculs, décisions, dispatch RabbitMQ	Persistance directe, JsonResponse
Manager	Crée/modifie les entités, appelle repository.save()	Règles métier, calculs, appels HTTP
Repository	Requêtes BDD (find, filter, count)	Modifier ou sauvegarder des entités

6.2 Exemple — ShipyardManager

```
// src/Manager/ShipyardManager.php
// → sait comment créer et persister une ShipyardQueue, rien de plus

class ShipyardManager
{
    public function __construct(
        private ShipyardQueueRepository $shipyardQueueRepository,
    ) {}

    public function createQueue(
        Shipyard $shipyard,
```

```
        string   $shipTypeSlug,
        int      $quantity,
        int      $buildTimeSeconds,
    ): ShipyardQueue {
        $queue = new ShipyardQueue()
            ->setShipyard($shipyard)
            ->setShipTypeSlug($shipTypeSlug)
            ->setQuantity($quantity)
            ->setQueuedAt(new \DateTime())
            ->setFinishAt(new \DateTime("+{$buildTimeSeconds} seconds"));

        $this->shipyardQueueRepository->save($queue, true);

        return $queue;
    }
}

// src/Service/Shipyard/ShipyardService.php
// → décide si on peut construire, calcule le temps, délègue la persistance

class ShipyardService
{
    public function buildShip(BuildShipDTO $dto, Planet $planet, Player $player):
ShipyardQueue
    {
        // Logique métier : vérification des prérequis
        $definition = $this->shipCatalog->get($dto->shipTypeSlug);
        $result = $this->requirementChecker->check($definition-
>unlockRequirements, $player, $planet);
        if (!$result->isSatisfied()) {
            throw new RequirementNotMetException($result);
        }

        // Logique métier : calcul du temps de construction
        $buildTime = $this->buildTimeCalculator->calculate(
            $definition,
            $planet->getShipyard()->getLevel(),
            $dto->quantity
        );

        // Délègue la persistance au Manager
        $queue = $this->shipyardManager->createQueue(
            $planet->getShipyard(), $dto->shipTypeSlug, $dto->quantity,
$buildTime
        );
```

```
        // Émet le message RabbitMQ
        $this->messageBus->dispatch(
            new ShipConstructionFinished($queue->getId()),
            [new DelayStamp($buildTime * 1000)]
        );

        return $queue;
    }
}
```

6.3 Les Managers et Integration/

Un Manager peut injecter un service d'Integration/ quand la persistance implique un service externe. Par exemple, ColonyManager peut injecter S3Service pour stocker une image liée à une planète — exactement comme DomainManager injectait S3 dans le projet précédent.

> *La règle reste la même : Integration/ ne connaît pas le jeu. Le Manager lui fournit les données à stocker/supprimer, Integration/ s'occupe du comment.*

7. src/Service/

Cœur de la logique métier. Organisé en sous-dossiers par domaine fonctionnel pour rester lisible. Chaque service a une responsabilité claire et délimitée.

```
Service/
├── Catalog/
│   ├── ShipCatalog.php
│   ├── BuildingCatalog.php
│   ├── TechnologyCatalog.php
│   ├── DefenseCatalog.php
│   └── CombatModifierCatalog.php
│
├── Colony/
│   ├── ResourceService.php
│   ├── BuildingService.php
│   ├── ColonizationService.php
│   └── DefenseService.php
│
├── Combat/
│   ├── CombatEventService.php
│   ├── CombatResolver.php
│   ├── CampResolver.php
│   ├── HitCalculator.php
│   └── TechnologyBonusResolver.php
│
├── Fleet/
│   ├── FleetService.php
│   ├── FleetMovementService.php
│   └── CargoService.php
│
├── Notification/
│   ├── CombatNotificationService.php
│   └── FleetNotificationService.php
│
├── Player/
│   ├── AllianceService.php
│   ├── GuildRelationService.php
│   └── RelationResolver.php
│
├── Requirement/
│   ├── RequirementChecker.php
│   └── ShipUnlockChecker.php
│
├── Research/
```

```
│   ├── ResearchService.php
│   └── ResearchTimeCalculator.php
│
├── Shipyard/
│   ├── ShipyardService.php
│   └── BuildTimeCalculator.php
│
└── Universe/
    ├── UniverseGenerator.php
    └── FogOfWarService.php
```

6.1 Détail par sous-dossier

Catalog/

Chargement et accès aux catalogues YAML. Ces services sont instanciés une seule fois au boot de Symfony (singleton). Ils scannent leur dossier respectif, parsent les fichiers YAML et exposent des méthodes get(slug) et all().

Fichier	Rôle
ShipCatalog.php	Charge tous les YAML de ships/. Expose get(slug): ShipDefinition et all().
BuildingCatalog.php	Idem pour buildings/. Expose aussi getRequirementsForLevel(slug, level).
TechnologyCatalog.php	Idem pour technologies/. Expose l'effet (unlock ou stat_bonus) de chaque techno.
DefenseCatalog.php	Idem pour defenses/.
CombatModifierCatalog.php	Charge combat_modifiers.yaml. Expose getModifier(attackerSlug, defenderSlug): float.

Colony/

Fichier	Rôle
ResourceService.php	Lazy calculation des ressources. Calcule le stock réel à la demande et persiste lors d'une consommation.
BuildingService.php	Upgrade d'un bâtiment. Vérifie les prérequis, déduit les ressources, planifie la fin via RabbitMQ.
ColonizationService.php	Colonisation d'une planète disponible. Consomme le vaisseau colonie.
DefenseService.php	Construction de défenses statiques sur une planète (via Forteresse).

Combat/

Fichier	Rôle
CombatEventService.php	Création du CombatEvent au premier attaquant. Gestion des arrivées pendant le timer. Déclenchement de la résolution à T+X.
CombatResolver.php	Résolution complète du combat : rounds, tirs, dégâts, destructions. Génère le CombatLog.
CampResolver.php	Détermine les camps (1v1, coalition, FFA) en interrogeant RelationResolver.
HitCalculator.php	Applique la formule de toucher (clamp 15-95) et les modificateurs de dégâts.
TechnologyBonusResolver.php	Calcule les bonus de stat actifs d'un joueur au moment du combat.

Fleet/

Fichier	Rôle
FleetService.php	Création d'une flotte, assignation d'une mission (attaque, défense, transport, orbite), gestion des statuts.
FleetMovementService.php	Calcul des distances (3 segments), temps de trajet, vérification de portée. Utilise les coordonnées X,Y et les positions de planètes.
CargoService.php	Calcul de l'hydrogène nécessaire, validation de la capacité de soute, chargement au départ, gestion du butin après combat.

Player/

Fichier	Rôle
AllianceService.php	Création et rupture des contrats d'alliance joueur↔joueur. Vérifie la compatibilité avec les relations inter-guildes.
GuildRelationService.php	Gestion des statuts entre guildes (neutre/alliés/ennemis). Rupture automatique des contrats incompatibles.
RelationResolver.php	Répond à la question : ces deux joueurs sont-ils alliés ? Consulté par CampResolver et les services de notification.

Requirement/

Services partagés, utilisés par ShipyardService, BuildingService et ResearchService.

Fichier	Rôle
RequirementChecker.php	Vérifie un RequirementSet (technologies + bâtiments + ships) pour un joueur et une planète. Retourne un RequirementResult.
ShipUnlockChecker.php	Vérifie récursivement si un vaisseau est débloqué (prérequis ships en cascade). Protège contre les cycles.

Research/

Fichier	Rôle
ResearchService.php	Lancement d'une recherche. Vérifie les contraintes (1 recherche active par planète, même techno non lancée ailleurs), planifie via RabbitMQ.
ResearchTimeCalculator.php	Calcule le temps de recherche selon le niveau de la technologie et le niveau du laboratoire de la planète.

Shipyard/

Fichier	Rôle
ShipyardService.php	Ajout d'une commande en queue. Vérifie les prérequis (RequirementChecker), déduit les ressources, émet le message RabbitMQ.
BuildTimeCalculator.php	Calcule le temps de construction selon le type de vaisseau, la quantité et le niveau du chantier.

Universe/

Fichier	Rôle
UniverseGenerator.php	Génération de la carte au démarrage du serveur. Crée galaxies, systèmes stellaires et planètes avec leurs positions et taux de production.
FogOfWarService.php	Gestion des découvertes (PlayerDiscovery). Enregistre les planètes découvertes lors d'un scan de sonde. Filtre les réponses API selon la visibilité du joueur.

Notification/

Fichier	Rôle
CombatNotificationService.php	Détermine qui notifier lors d'une attaque (membres guilde + contrats directs). Applique les règles de visibilité de l'identité de l'attaquant.
FleetNotificationService.php	Notifie l'arrivée d'une flotte, la fin d'un transport, le retour après combat.

8. src/Message/ et src/MessageHandler/

Symfony Messenger avec RabbitMQ. Message/ contient les DTOs (objets simples transportés dans la queue). MessageHandler/ contient les consumers qui traitent ces messages.

Message (DTO)	Émis par	Handler — action
ShipConstructionFinished.php	ShipyardService	Ajoute les vaisseaux à FleetShip, notifie le joueur via Mercure.
ResearchFinished.php	ResearchService	Incrémente PlayerTechnology, notifie le joueur.
FleetArrived.php	FleetMovementService	Change le statut de la flotte, crée ou rejoint un CombatEvent si attaque.
CombatResolve.php	CombatEventService	Déclenche CombatResolver à T+X, persiste le log, notifie les participants.

```
// Exemple de Message — objet simple, pas de logique
// src/Message/ShipConstructionFinished.php
class ShipConstructionFinished
{
    public function __construct(
        public readonly int $shipyardQueueId,
    ) {}
}

// Exemple de Handler — orchestre les services, pas de logique propre
// src/MessageHandler/ShipConstructionFinishedHandler.php
class ShipConstructionFinishedHandler implements MessageHandlerInterface
{
    public function __invoke(ShipConstructionFinished $message): void
    {
        $queue = $this->shipyardQueueRepository->find($message->shipyardQueueId);
        $this->shipyardService->completeConstruction($queue);
        $this->fleetNotificationService->notifyConstructionComplete($queue);
    }
}
```

9. src/Integration/

Services d'infrastructure externe. Ces services ne connaissent pas le jeu — ils savent uniquement parler à un service tiers. Ils sont remplaçables et testables via mock indépendamment du reste.

```
Integration/
├── Aws/
│   └── S3Service.php            # Upload, download, suppression de fichiers sur
S3
├── Discord/
│   └── DiscordService.php       # Envoi de messages sur un channel Discord
└── Mercure/
    └── MercurePublisher.php     # Publication d'événements SSE via Mercure
```

Fichier	Rôle
S3Service.php	Abstraction de l'AWS SDK. Expose upload(file, path), download(path), delete(path). Ne connaît pas ce qu'on stocke.
DiscordService.php	Appel HTTP vers l'API Discord. Expose send(channelId, message). Les services de Notification/ lui fournissent le contenu.
MercurePublisher.php	Abstraction du hub Mercure. Expose publish(topic, data). Utilisé par les services de Notification/.

> *La distinction clé : CombatNotificationService (dans Service/Notification/) sait QUOI envoyer et À QUI. DiscordService (dans Integration/) sait comment envoyer un message à Discord. Ces deux responsabilités ne se mélangent pas.*

10. src/ValueObject/

Objets immuables sans persistance en BDD. Un Value Object représente une valeur métier — deux instances avec les mêmes données sont équivalentes. Ils n'ont pas d'identifiant, ne changent pas après construction.

```
ValueObject/
├── Definition/                  # Objets immuables issus des catalogues YAML
│   ├── ShipDefinition.php       # Stats d'un type de vaisseau (hp, precision,
costs...)
│   ├── BuildingDefinition.php   # Définition d'un type de bâtiment
│   ├── TechnologyDefinition.php # Définition d'une technologie (effet,
prérequis)
│   ├── DefenseDefinition.php    # Définition d'un type de défense
│   └── RequirementSet.php       # Ensemble de prérequis (techs + buildings +
ships)
│
└── Requirement/                 # Résultats de vérification de prérequis
    ├── RequirementResult.php    # Résultat global (satisfait ou non + liste des
échecs)
    ├── MissingTechnology.php    # Technologie manquante (slug, niveau requis,
niveau actuel)
    ├── MissingBuilding.php      # Bâtiment manquant (slug, niveau requis, niveau
actuel)
    └── MissingShipUnlock.php    # Vaisseau prérequis non débloqué
```

9.1 Différence entre ValueObject et Entity

	Entity	ValueObject
Persistance	Oui — table en BDD	Non — jamais en BDD
Identité	Oui — ID unique	Non — défini par ses valeurs
Mutabilité	Oui — peut changer	Non — immuable
Exemple	Fleet, PlanetBuilding	ShipDefinition, RequirementResult

11. src/Validator/DTO/

DTOs de validation des données entrantes (requêtes HTTP). Chaque DTO correspond à une action du joueur. Il désérialise le payload brut, normalise les valeurs et déclare des contraintes de validation via les attributs Symfony Validator.

```
Validator/
└── DTO/
    ├── Fleet/
    │   ├── SendFleetDTO.php          # Envoi d'une flotte (destination,
composition, vitesse)
    │   └── UpdateFleetSpeedDTO.php   # Modification de la vitesse en cours de
route
    ├── Shipyard/
    │   └── BuildShipDTO.php          # Commande de construction (slug, quantité)
    ├── Research/
    │   └── StartResearchDTO.php      # Lancement d'une recherche (slug
technologie)
    ├── Colony/
    │   ├── UpgradeBuildingDTO.php    # Upgrade d'un bâtiment
    │   └── BuildDefenseDTO.php       # Construction de défenses
    └── Player/
        ├── CreateAllianceDTO.php     # Proposition d'alliance joueur↔joueur
        └── GuildRelationDTO.php      # Déclaration de statut entre guildes
```

10.1 Structure d'un DTO

Chaque DTO suit la même structure :

- Constructeur qui reçoit le payload brut (array) et normalise les valeurs
- Propriétés publiques typées
- Méthodes de validation avec #[Constraints\Callback] et messages d'erreur traduits

```
// src/Validator/DTO/Shipyard/BuildShipDTO.php
class BuildShipDTO
{
    public ?string $shipTypeSlug;
    public ?int    $quantity;

    public function __construct(array $payload)
    {
        $this->shipTypeSlug =
Normalizer::normalizeString($payload['ship_type_slug'] ?? null);
        $this->quantity     = isset($payload['quantity']) ? (int)
$payload['quantity'] : null;
    }
```

```
    #[Constraints\Callback]
    public function validateShipTypeSlug(ExecutionContextInterface $context):
void
    {
        if (!$this->shipTypeSlug) {
            $context->buildViolation('shipyard.error.slug.required')
                ->atPath('ship_type_slug')
                ->setTranslationDomain('api')
                ->setCause(['httpStatus' => Response::HTTP_BAD_REQUEST])
                ->addViolation();
        }
    }
}
```